

Option Big Data

Oral final

Introduction:

Hello everyone, today I will present my Black-Scholes option pricer project. This project is part of my course, and the goal was to create a small application to calculate option prices and their sensitivities, called Greeks.

The main objective of the application is to allow a user to compute the theoretical price of a call or a put option, visualize how the price changes when parameters change, and understand the impact of each parameter through Greeks like Delta, Gamma, Vega, Theta, and Rho.

In this presentation, I will first explain the mathematical background, then show how I implemented it in Python, and finally demonstrate the application with a practical example. I will finish with a short conclusion and possible improvements.

Mathematics:

Now, I will explain the mathematical background of the Black-Scholes model.

First, a quick reminder about options. A **call option** gives the right to buy an asset at a fixed price, called the strike, before a certain date. A **put option** gives the right to sell the asset at the strike price before the expiry date. The main parameters are the spot price, strike price, maturity, risk-free interest rate, and volatility.

The Black-Scholes model allows us to calculate the theoretical price of these options. It is based on some assumptions: the asset price follows a **geometric Brownian motion**, markets are efficient, there are no transaction costs, and the risk-free rate is constant.

The formulas for the call and put options use two values called **d1** and **d2**. Using these, the **call price** is computed as the spot price times the cumulative normal of d1, minus the discounted strike price times the cumulative normal of d2. The **put price** is similar but adapted for selling.

We also calculate **Greeks**, which measure sensitivities. Delta shows how the price changes with the underlying asset. Gamma shows the sensitivity of Delta. Vega measures the impact of volatility. Theta is the sensitivity to time, and Rho measures the impact of interest rates.

These Greeks help to understand the behavior of the option price when market conditions change.

Now about the part "Portfolio Manager". The idea is to minimize the net delta of our portfolio.

The delta is the sensitivity of the price of the option about the price of the stock.

The net delta of my portfolio is the ponderate sum of my different delta (call, put and asset).

With an Optimization Heuristic I can find the K and the T to minimize it.

To finish, to represent our PNL we take the assumption that our future price follows a normal distribution around S_0 .

Warning, it is an approximation and not the exact PNL.

Informatic:

Now, I will explain how I implemented the Black-Scholes pricer in Python.

I used **Python** because it is simple and efficient for numerical calculations. The main libraries are:

- **NumPy** for numerical operations,
- **SciPy** to use the normal distribution for the formulas,
- **Plotly** for interactive charts,
- **yfinance** to fetch real market prices, and
- **Streamlit** to create the web interface.

The code is organized in functions. First, I calculate **d1 and d2**, then I compute the **option price** for a call or a put. Another function calculates the **Greeks**, either for a single set of parameters or for a range of values, which allows us to make interactive plots.

The interface is designed to be user-friendly. On the sidebar, the user can select the type of option, enter the parameters, and choose which parameter to vary for the plots. The main page shows the **theoretical price** and the **Greeks**, as well as an interactive plot that updates automatically when the user changes the inputs.

This structure makes the application both educational and practical for exploring option behavior in real-time.

I will finish with two blocks of code about the page "Portfolio Project".

At left, you can see the block about my heuristic optimization we just choose some grids about T and K and we try to minimize the net delta of our portfolio.

At right, you can see the block about the PNL Simulation. It's just a simple plot with a normal distribution wich is around S_0 .

Presentation:

Now, I will show a practical example using the application.

Let's take a **call option** with a spot price of 100, a strike of 100, one year to maturity, a risk-free rate of 5%, and volatility of 20%. The application calculates the **theoretical price** and the **Greeks** instantly.

We can also change the parameters, for example the spot price or the volatility, and the **interactive plot updates automatically**, showing how the price and Greeks change.

This allows the user to quickly understand the impact of each parameter and explore different market scenarios in real-time.

We can do the same with a **put option**. The interface is simple, intuitive, and very interactive.

Conclusion:

To conclude, this project allowed me to create a complete **Black-Scholes pricer** with Python and Streamlit. We can calculate **option prices** and **Greeks**, visualize their behavior, and explore different market scenarios interactively.

Through this project, I learned not only about the **mathematics of option pricing**, but also about **programming, data visualization, and building a user-friendly interface**.

For future improvements, we could include more advanced models, like **stochastic volatility or Monte Carlo simulations**, and improve the interface with more customization and features for professional use.

Thank you for your attention.